Merge Agent — Pseudocode & Semantic Diff Algorithm

Purpose: Provide detailed pseudocode and operational rules for the Merge Agent, responsible for computing diffs between child and parent nodes, producing human-friendly semantic summaries, enforcing merge policy, applying approved merges, creating snapshots, handling conflicts, and propagating staleness.

Goals & Responsibilities

- Compute precise textual diffs (unified diff / JSON Patch) between parent and child node content.
- Produce a semantic (natural-language) summary of what the changes mean — not just the text differences.
- Enforce merge policy (sequential merges for MVP) and detect merge conflicts.
- Create version snapshots before applying any change, and store commit metadata.
- Apply approved merges atomically, with rollback support.
- Mark dependents as `stale` after parent update and emit events.
- Maintain audit logs for merges and enable easy rollback.

High-level flow

1. `compute_merge(child_id)` is invoked when a child requests a merge.
2. Load child content and parent current content.
3. Compute a textual diff (unified diff and JSON Patch where appropriate).
4. Call `semantic_summarizer` (LLM) with the parent/child snippets + diff to produce a short NL summary of intent and impact.
5. Detect conflicts against other pending merges (overlapping patches); if conflicts exist, create a `merge_conflict` child node and queue for user resolution.
6. Return `MergePreview` (textual diff, semantic summary, estimated impact) to the frontend.
7. If user approves, `apply_merge(child_id, approver)` is called:
8. Create snapshot (commit) of parent.
9. Apply patch to parent in a transaction.
10. Update `merge_request` status to `approved` and write `Version` record.
11. Mark dependents stale and emit events.
12. Return success and commit_id.

Data formats

- **Textual diff:** unified diff (standard `diff -u`) for text files/code. For structured documents (JSON/YAML), generate JSON Patch (RFC 6902).

- **Semantic summary:** short paragraph (2–4 sentences) describing what changed, why it matters, which fields/components are affected, and suggested follow-ups.
- **MergeRequest record:** `{ merge_id, child_id, parent_id, text_diff, json_patch, diff_summary, status, created_at, requested_by }`.

Pseudocode — helper utilities

```
def load_node_content(node_id: str) -> str:
    # fetch latest content snapshot for node (e.g., combined message text or code field)
    pass

def compute_unified_diff(a: str, b: str) -> str:
    # returns unified diff as string (using difflib.unified_diff or GNU diff wrapper)
    pass

def compute_json_patch(a_json: dict, b_json: dict) -> List[dict]:
    # returns JSON Patch operations (RFC 6902)
    pass

def semantic_summarizer(parent_snippet: str, child_snippet: str, diff_text: str) -> str:
    # call LLM to summarize changes semantically
    # prompt should ask for: what changed?, why it matters?, potential downstream effects, suggested tests
    pass

def create_snapshot(node_id: str, author: str, reason: str) -> str:
    # snapshot current node state and return commit_id
    pass

def apply_json_patch_to_content(content: dict, patch: List[dict]) -> dict:
    # apply JSON patch safely
    pass
```

Pseudocode — compute_merge(child_id)

```
def compute_merge(child_id: str) -> MergePreview:
    child = GraphDB.get_node(child_id)
    parent_id = GraphDB.get_parent_of_child(child_id)
    parent = GraphDB.get_node(parent_id)
```

```
    # 1. Extract canonical content from nodes (could be aggregated text or
structured docs)
    parent_content = load_node_content(parent_id)
    child_content = load_node_content(child_id)

    # 2. Compute text diff
    text_diff = compute_unified_diff(parent_content, child_content)

    # 3. If contents are structured (JSON / YAML), also compute json_patch
    if is_structured(parent_content) and is_structured(child_content):
        parent_json = parse_structured(parent_content)
        child_json = parse_structured(child_content)
        json_patch = compute_json_patch(parent_json, child_json)
    else:
        json_patch = None

    # 4. Conflict detection vs other pending merges
    overlapping_merges = MergeDB.find_pending_merges_touching_parent(parent_id)
    for m in overlapping_merges:
        if m.child_id == child_id:
            continue
        if patches_overlap(m.text_diff, text_diff) or
patches_conflict(m.json_patch, json_patch):
            # create a conflict node that references both merges
            conflict_node = create_merge_conflict_node(parent_id, [m.merge_id,
child_id])
            return MergePreview(conflict=True,
conflict_node_id=conflict_node.id)

    # 5. Semantic summarization (call LLM)
    # send parent snippet, child snippet, and text_diff to LLM summarizer
    summary = semantic_summarizer(parent_snippet=parent_content,
child_snippet=child_content, diff_text=text_diff)

    # 6. Estimate impact: count changed lines, identify affected subcomponents
via heuristics
    impact = estimate_impact(text_diff)

    # 7. Persist MergeRequest record with status 'pending'
    merge_record = MergeDB.create({
        'child_id': child_id,
        'parent_id': parent_id,
        'text_diff': text_diff,
        'json_patch': json_patch,
        'diff_summary': summary,
        'status': 'pending',
        'created_at': now(),
```

```
        'requested_by': child.metadata.requested_by
    })

    return MergePreview(merge_id=merge_record.merge_id, text_diff=text_diff,
json_patch=json_patch, diff_summary=summary, impact=impact)
```

Pseudocode — apply_merge(child_id, approver)

```
def apply_merge(child_id: str, approver: str) -> ApplyResult:
    # 1. Fetch merge record
    merge = MergeDB.find_by_child(child_id)
    if not merge or merge.status != 'pending':
        raise MergeError('Invalid merge state')

    parent_id = merge.parent_id

    # 2. Create parent snapshot
    commit_id = create_snapshot(parent_id, author=approver, reason=f'Applying
merge {merge.merge_id}')

    # 3. Apply patch in a transaction
    try:
        with DB.transaction():
            parent_content = load_node_content(parent_id)

            if merge.json_patch:
                parent_json = parse_structured(parent_content)
                new_json = apply_json_patch_to_content(parent_json,
merge.json_patch)
                new_content = serialize_structured(new_json)
            else:
                new_content = apply_text_patch(parent_content, merge.text_diff)

            # 4. Update parent node content and updated_at
            GraphDB.update_node_content(parent_id, new_content)

            # 5. Update merge record status
            MergeDB.update(merge.merge_id, status='approved',
applied_by=approver, applied_at=now(), applied_commit=commit_id)

            # 6. Create Version record linked to parent
            VersionService.create_version(parent_id, commit_id,
merge.diff_summary, author=approver)
```

```
    except Exception as e:
        # rollback handled by transaction; mark merge as failed
        MergeDB.update(merge.merge_id, status='failed', error=str(e))
        raise

    # 7. Mark dependents stale (emit background job)
    AgentBus.publish('node.updated', {'node_id': parent_id, 'commit_id':
commit_id})

    return ApplyResult(success=True, commit_id=commit_id)
```

Conflict handling policy (MVP)

- **Sequential merges:** merges are applied in queue order. If an earlier merge modifies a region a later merge also wants to change, `compute_merge` will detect overlap when the later merge is evaluated and return a `merge_conflict` preview.
- **Conflict resolution node:** create a special `merge_conflict` child node that includes both conflicting diffs and asks the user to resolve manually via UI. The UI will show combined diffs and allow editing a resolution which becomes a new child node to apply.
- **Automatic merging:** Not implemented in MVP. Future: intelligent 3-way merge using semantic analysis if conflicts are non-overlapping or reconcilable.

Semantic summarizer prompt templates

System Prompt:

```
You are a precise software editor. Given the original content and an updated
content and the textual diff between them, produce a 3-bullet summary
describing:
1) What changed (high-level)
2) Why it matters / potential downstream effects
3) Suggested tests or validation steps for the change
Be concise and use plain language.
```

User Prompt (example):

```
Parent content (short):
<parent_snippet>

Child content (short):
<child_snippet>
```

```
Diff:
<unified_diff>

Produce the 3-bullet summary as requested.
```

Edge cases & fallbacks

- **Large diffs:** If diff is too large for one summarization call, chunk the diff and ask the LLM to summarize chunks, then synthesize a top-level summary.
- **Binary/Blob updates:** If content is binary (images, binaries), store pointers and include a descriptive summary; require manual approval.
- **Structured vs unstructured:** Prefer JSON Patch for structured documents to preserve semantics. For unstructured text, use unified diff.

Observability & Metrics

- `merge_requests_created`, `merge_requests_approved`, `merge_conflicts_created`, `average_merge_time_ms`, `semantic_summary_latency_ms`, `apply_merge_success_rate`.
- Audit log entries for: who requested, who approved, commit_id, timestamp, diff sizes.

Tests & Validation

- Unit tests for: `compute_unified_diff`, `compute_json_patch`, `apply_json_patch_to_content`.
- Integration tests simulating: concurrent merges, conflict creation, apply_merge with rollback on failure.
- E2E tests for the UI merge flow: create child, compute merge, show preview, approve, verify parent updated and dependents stale.

Rollback strategy

- Because we create a snapshot before applying merges, rollbacks are implemented by restoring the previous version commit and setting a new version record documenting the rollback. Also mark the merge request as `reverted`.

End of Merge Agent pseudocode & semantic diff algorithm document.